Lab 3

Some starters and resources

Primitive Types in OpenGL:

GL_POINTS: Points
GL_LINE_STRIP: Continuous non looping line
GL_LINE_LOOP: A continuous looping line
GL_LINES: only 2 points make a line so discontinuous lines
GL_LINE_STRIP_ADJACENCY: Like a line strip but we neglect the first and the last points
GL_LINES_ADJACENCY: Same as lines but ignores the start and end vertices in the drawing
GL_TRIANGLE_STRIP: Draws continuously connect triangles
GL_TRIANGLE_FAN: It's a circular triangle looking thing that we can use to create complex objects such as circles and quads
GL_TRIANGLES: Draws a triangle with every 3 vertices
GL_TRIANGLE_STRIP_ADJACENCY: Like a triangle strip but every other vertex ignored
GL_TRIANGLES_ADJACENCY: Same as triangles but every 6 points is a separate group, and only 3 of them (every other one) are actually drawn
GL_PATCHES: Some vertices passed for tessellation, something like the Bezier curve, terrain maps. It requires some control points that help us guide the shapes through.

Mouse and Keyboard functions

https://www.youtube.com/watch?v=L2aiuDDFNIk
https://www.glfw.org/documentation.html (specifically: https://www.glfw.org/docs/latest/input_guide.html)

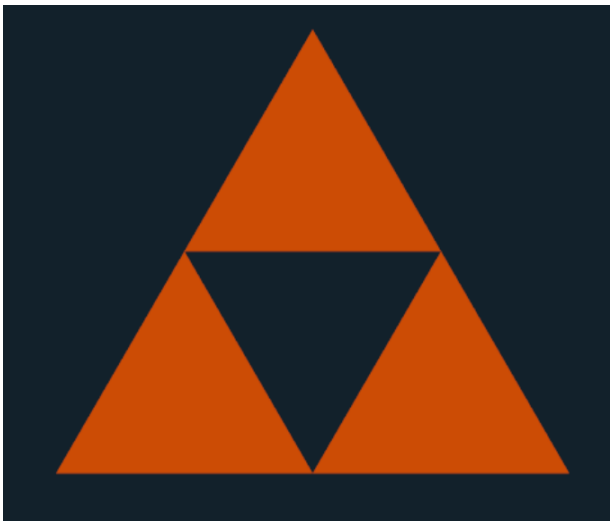
VBOs, VAOs, EBOs:

https://www.youtube.com/watch?v=KG9ZXKaJWwY

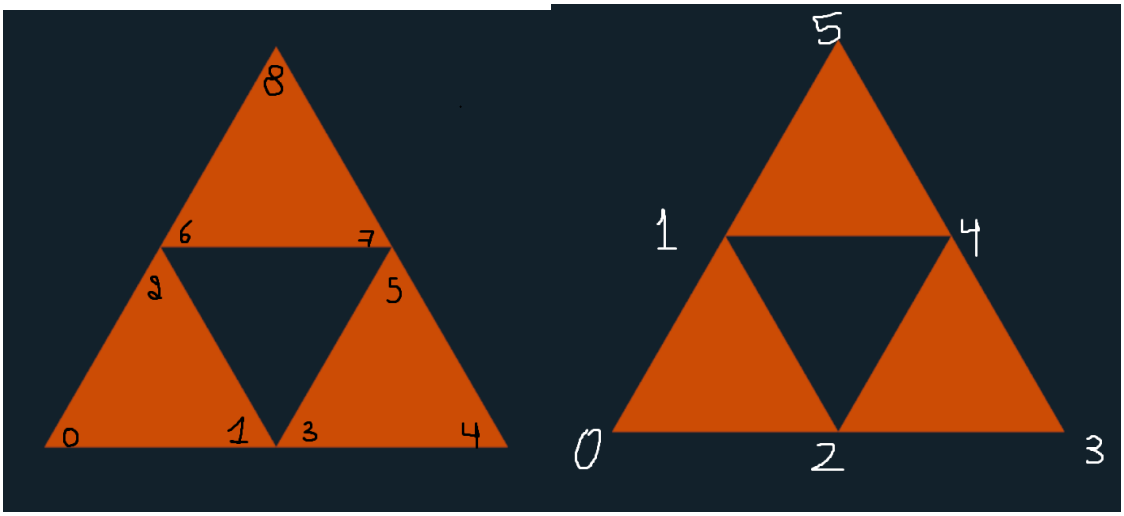
Main discussion points:

EBO:

An EBO is an element buffer object, which can also be referred to as an index buffer. Its main usage is aiding us in rendering shapes more easily like this!



How it works essentially is instead of defining 9 vertices for triangles ABC, DEF, and XYZ we merge common vertices and only get 6 vertices.



To tell OpenGL about this, we define only those 6 indices

```
std::vector<float> vertices = {
    //Positions             //Colors
    -0.5f, -0.5f, 0.0f,     1.0f, 0.0f, 0.0f, //0
    -0.25f, 0.0f, 0.0f,     0.5f, 0.0f, 0.5f, //1
     0.0f, -0.5f, 0.0f,     0.5f, 0.5f, 0.0f, //2
     0.5f, -0.5f, 0.0f,     0.0f, 1.0f, 0.0f, //3
     0.25f, 0.0f, 0.0f,     0.0f, 0.5f, 0.5f, //4
     0.0f, 0.5f, 0.0f,      0.0f, 0.0f, 1.0f  //5
};
```

And then we define the index buffer which tells OpenGL how to traverse to draw the shape (BEWARNED THE ORDER IS VERY VERY VERY IMPORTANT OTHERWISE YOUR TRIANGLE WILL LOOK DISTORTED)

```
GLuint indices[] = {
    0, 2, 1,
    2, 3, 4,
    4, 5, 1
};
```

Now that we're done, all you need to do is define another unsigned int for GL (GLuint EBO) and bind it and assign it and you're good! (You'll find it in the attached code)

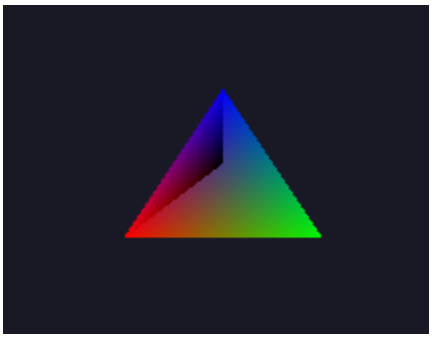
Now enjoy your super duper cool and awesome triangles!

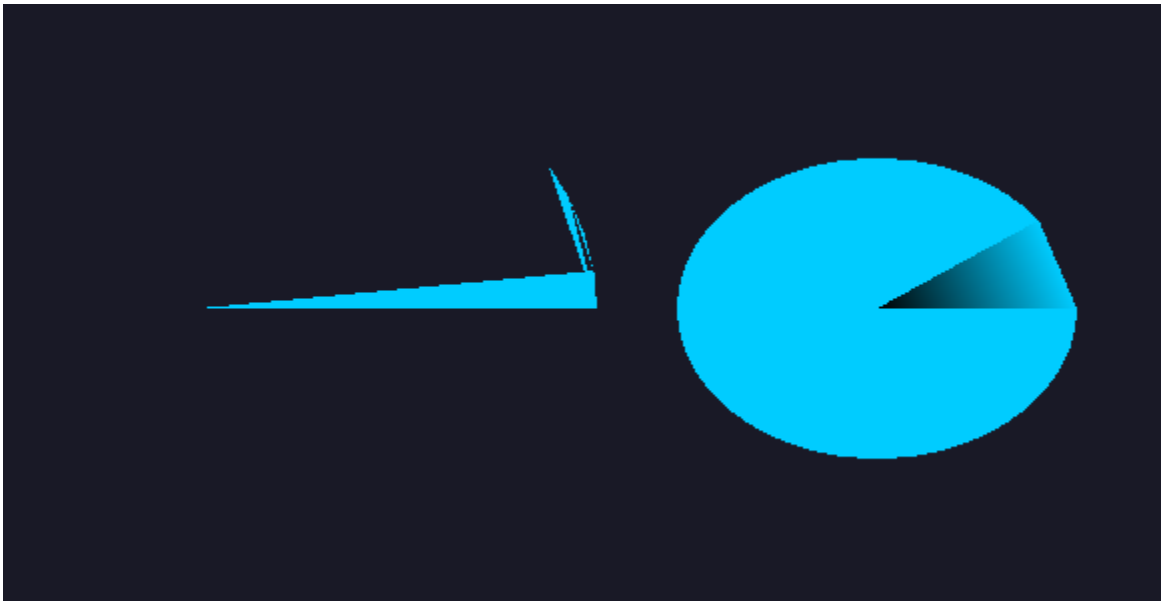
Global VBOs:

You can add more objects and vertices into your VBOs, all you have to do is… PUT IN THE VERTICES DATA! (WAW!)
But what about when I want to render it? Guess what? (WHAT?!)
glDrawArrays lets you do it!! You just gotta point to your first index and your size! And tell us your primitive!

void glDrawArrays(GLenum mode, GLint first, GLsizei count);

Alternatively you could use more than a singular VBO and VAO for different objects, during this case you treat them as an array, and make sure to define each object's VBO and EBO in the VAO binding before moving on to the other one, or you will have murdered your project!





(Misshapen graphics due to human error)

Mouse and Keyboard functions!

The long awaited functions. How do we accept user input??
Well, dear reader, like everything I've discussed so far… you have *drum rolls* OPTIONS!
You've got two ways to go at it
Now our lovely library GLFW has offered us something unique. Setting up our own call back functions! Which means of course that we have to code them.

```
glfwSetKeyCallback(window, keyCallback);
glfwSetMouseButtonCallback(window, mouseButtonCallback);
glfwMakeContextCurrent(window);
```

Now to code them we need to give them the method signature and attach our code inside:

```
void keyCallback(GLFWwindow* window, int key, int scancode, int action, int mods)
{
```

The window is obviously the window we're currently using.
The key is the key being pressed.
The scancode is something more platform specific and unique so we don't often use it, but it is a parameter you get. Mainly for unknown keys since you have that option and it's pretty nifty!
The action is whether this is a single click, continuous hold or release option.
The mods is when you want to have something special like: "ctrl + A" so you specify your mod is ctrl.
This is an example of it in usage and how we can code the body of the function.

```
if (key == GLFW_KEY_ESCAPE && action == GLFW_PRESS)
    glfwSetWindowShouldClose(window, true);

if (key == GLFW_KEY_A && action == GLFW_REPEAT) posX -= moveSpeed;
if (key == GLFW_KEY_D && action == GLFW_REPEAT) posX += moveSpeed;
if (key == GLFW_KEY_W && action == GLFW_REPEAT) posY += moveSpeed;
if (key == GLFW_KEY_S && action == GLFW_REPEAT) posY -= moveSpeed;
```

The mouse function is pretty similar except it doesn't have the scan code, and mouse buttons are usually normally only 3.

```
void mouseButtonCallback(GLFWwindow* window, int button, int action, int mods)
{
    if (button != GLFW_MOUSE_BUTTON_LEFT) //The click, not the right click
        return;

    if (action == GLFW_PRESS)
        isMousePressed = true;
    else if (action == GLFW_RELEASE) //Here is release in action and the reset logic WEEE
        isMousePressed = false;
}
```

Alternatively… you can also use this nifty function

```
//processInput(window);
```

Which has a body you code!!

```
//Optional keyboard
/* void processInput(GLFWwindow window)
{
    if (glfwGetKey(window, GLFW_KEY_ESCAPE) == GLFW_PRESS)
        glfwSetWindowShouldClose(window, true);

    if (glfwGetKey(window, GLFW_KEY_LEFT) == GLFW_PRESS) posX -= moveSpeed;
    if (glfwGetKey(window, GLFW_KEY_RIGHT) == GLFW_PRESS) posX += moveSpeed;
    if (glfwGetKey(window, GLFW_KEY_UP) == GLFW_PRESS) posY += moveSpeed;
    if (glfwGetKey(window, GLFW_KEY_DOWN) == GLFW_PRESS) posY -= moveSpeed;
}
```

Including the mouse function the cool thing that happens is that you can highlight the triangle when hovering and highlight the circle as long as you're clicking on the mouse!

Multiple VBOs and VAOs:

You will find the 2vs code attached, this is simply to show how you can implement more than one VBO and VAO in your code for cleanup, it's unnecessary in our current code size and stage but good practice for future reference.

Shader updates

In your Fragment Shader Source code, you will find this lovely little piece of code attached:

```
void main()
{
    vec3 finalColor = mix(vColor, vec3(1.0, 1.0, 0.0), u_Highlight);
    FragColor = vec4(finalColor, 1.0);
}
)";
```

What it does it utilizing the mix function which works to blend two colors, the third parameters defines how heavily the second colors affects the first so sort of like an alpha channel but not really.

The u_Highlight value is an int that gets updated on mouse button click and when we're colliding with the triangle.

Collision function:

We use two functions here for the collision: the first one is tri sign. What it essentially does is it tells us which side of a certain line this point we have lies on by calculating the cross product. So we can tell if we're on the left or the right or if we're on the line itself (that's why the very edge doesn't really cause detection).

```
float triSign(glm::vec2 p1, glm::vec2 p2, glm::vec2 p3)
{
    return (p1.x - p3.x) * (p2.y - p3.y) - (p2.x - p3.x) * (p1.y - p3.y);
}
```

And the second in pointInTriangle, we utilize the previously made tri sign to actually check if we're in the triangle.

```
bool pointInTriangle(glm::vec2 pt, glm::vec2 a, glm::vec2 b, glm::vec2 c)
{
    float d1 = triSign(pt, a, b);
    float d2 = triSign(pt, b, c);
    float d3 = triSign(pt, c, a);

    bool hasNeg = (d1 < 0) || (d2 < 0) || (d3 < 0);
    bool hasPos = (d1 > 0) || (d2 > 0) || (d3 > 0);

    return !(hasNeg && hasPos);
}
```